

TI C2000 - ESP32 UART (SCI) Haberleşmesi

1. Görevin Amacı

Görevin Kapsamı: Bu görev; sistemin mikrodenetleyici (TI C2000) ile Edge cihazı (ESP32) arasındaki deterministik veri köprüsünün kurulmasını amaçlar. Bu hat, sadece basit bir veri aktarım kanalı değil; sistemin ana şebekeden kopup "AdaModu"na geçmesini, bulut sistemlerine (MQTT) veri basmasını ve internetin olmadığı afet anlarında **LoRa Mesh** ağına otomatik geçiş yapmasını sağlayan ana sinir sistemidir.

Temel Hedefler:

- **Telemetri Köprüsü:** TI C2000 tarafından 100 kHz hızında okunan hassas güç verilerinin (Gerilim, Akım, Frekans), ESP32 üzerinden dış dünyaya (Wi-Fi veya LoRa Mesh) asenkron olarak iletilmesi.
- **Haberleşme İzolasyonu:** İşlemci darboğazlarını önlemek adına, karmaşık ağ protokollerinin (LoRa Mesh Routing, JSON Parsing, MQTT Stack) TI C2000'den arındırılarak tamamen ESP32 Hub'ı üzerinde koşturulması.
- **Afet Direnci (Hybrid Connectivity):** Şebeke ve internetin çöktüğü senaryolarda, UART hattından gelen verilerin ESP32 tarafından otomatik olarak LoRa Mesh moduna yönlendirilerek kritik düğümlere (Hastane, AFAD vb.) ulaştırılmasının sağlanması.
- **Sistem Güvenliği (Fail-Safe):** Donanım ve yazılım katmanları arasında bir "Watchdog" (Yaşam Sinyali) mekanizması kurularak, haberleşme kopması durumunda sistemin otonom olarak güvenli ada moduna geçirilmesi.

2. Yazılım İskeleti (C/C++)

TI C2000 tarafında koşturulacak olan bu algoritma; sensör verilerini toplama, veriyi "Mesh Uyumlu" JSON paketine dönüştürme ve ESP32'den gelen "Yaşam Sinyali"ni (Heartbeat) takip etme görevlerini üstlenir.

- Telemetri ve JSON Paketleme Fonksiyonu

Bu fonksiyon, okunan analog verileri ESP32'nin ve dolaylı olarak LoRa Mesh ağının anlayabileceği karakter dizisine çevirir:

// Veriyi JSON formatında ESP32'ye (LoRa/WiFi Hub) gönderir

```
void send_telemetry_to_hub() {  
    // 1. ADC Değerlerini Oku  
    float v_bar = AdcaResultRegs.ADCRESULT0 * SCALE_FACTOR_V;  
    float i_yuk = AdcbResultRegs.ADCRESULT0 * SCALE_FACTOR_I;  
    float f_grid = calculate_frequency();  
    // 2. Durum Tespiti (Normal/Islanding)  
    char* status = (f_grid < 48.5) ? "ISLANDING" : "NORMAL";  
    // 3. LoRa Mesh Uyumlu JSON Paketi  
    char payload[150];  
    sprintf(payload, "{\"id\":\"E1\",\"v\":%.1f,\"f\":%.2f,\"s\":\"%s\",\"target\":  
    [\"C1\",\"C2\"]}\n", v_bar, f_grid, status);  
    // 4. UART (SCI) Üzerinden Gönderim  
    sci_send_string(payload); }
```

- **Komut Yakalama ve Watchdog (Heartbeat) Mantığı**
Sistemin otonom güvenliğini sağlayan, ESP32'den gelen komutları ve "hayattayım" sinyalini dinleyen kesme (interrupt) bloğudur:

```
// Global Değişkenler
volatile int hub_heartbeat_counter = 300; // 3 saniyelik tolerans (10ms döngü için)

// UART üzerinden gelen verileri işler
void process_incoming_hub_data() {
    if (SciaRegs.SCIRXST.bit.RXRDY) {
        char cmd = SciaRegs.SCIRXBUF.all;

        // ESP32'den gelen "Yaşam Sinyali" (Heartbeat)
        if (cmd == 'H') {
            hub_heartbeat_counter = 300; // Sayaç resetlenir
        }

        // Karar Motorundan gelen Ada Modu/Röle Komutu
        if (cmd == '1') {
            GpioDataRegs.GPASET.bit.GPIO12 = 1; // Fiziksel Röle Tetiklenir
        }
    }
}

// Ana Timer Kesmesi (10ms)
interrupt void timer_isr(void) {
    hub_heartbeat_counter--;

    // Haberleşme Hub'ı (ESP32) çökerse veya kablo koparsa:
    if (hub_heartbeat_counter <= 0) {
        // İnisiyatif al ve sistemi güvenli ada moduna (Islanding) geçir
        GpioDataRegs.GPASET.bit.GPIO12 = 1;
        error_log("HABERLESME_KOPUKLUGU_OTONOM_ISOLASYON");
    }
}
```

Yazılımlara ilişkin açıklamalar:

- **JSON Yapısı:** Karakter dizisi içine gömülen target dizisi, ESP32 üzerindeki LoRa modülünün bu veriyi hangi mesh düğümlerine (C1, C2) yönlendireceğini belirler.
- **Asenkron Yapı:** sprintf ve sci_send işlemleri, PWM kesmelerini engellemeyecek şekilde düşük öncelikli bir döngüde çalıştırılır.
- **Deterministik Güvenlik:** hub_heartbeat_counter mantığı, üst katman yazılımı (CE) çökse dahi donanımın (EE) kendi güvenliğini sağlayabilmesini garanti eder.

3. Teknik Gereksinimler ve Veri Sözleşmesi (JSON)

Bu bölüm, iki işlemci arasındaki fiziksel bağın parametrelerini ve LoRa Mesh ağına basılacak verinin dijital yapısını tanımlar.

- Fiziksel Katman ve Pin Eşleşmesi

Haberleşmenin donanımsal düzeyde kayıpsız gerçekleşmesi için kullanılan pin konfigürasyonu aşağıdadır:

İşlev	TI C2000 Pini	ESP32 Pini	Teknik Detay
Veri Gönderim (TX)	SCITXDA (GPIO28)	RX0 (GPIO3)	3.3V TTL, 115200 bps
Veri Alım (RX)	SCIRXDA (GPIO29)	TX0 (GPIO1)	3.3V TTL, 115200 bps
Haberleşme Güvenliği	GPIO12 (Röle)	-	Watchdog Tetikleyici

- LoRa Mesh Uyumlu Veri Protokolü (JSON)

TI C2000, sensörlerden okuduğu verileri ESP32'ye gönderirken, Beril'in tasarladığı LoRa Mesh yönlendirme (routing) kurallarına uygun bir JSON sözleşmesi kullanır. Bu sayede ESP32, internet koptuğunda veriyi modifiye etmeden doğrudan LoRa modülüne aktarabilir.

Örnek Veri Paketi:

```
{
  "id": "E1",      // Enerji Düğümü Kimliği
  "v": 580.4,      // Bara Gerilimi (Volt)
  "f": 49.85,      // Şebeke Frekansı (Hz)
  "s": "ISLANDING", // Sistem Durumu (Normal/Ada Modu)
  "target": ["C1"], // Mesh Ağındaki Hedef (Hastane/Kritik Yük)
  "hop": 0         // Sıçrama Sayısı (Beril'in algoritması için)
}
```

- Hibrit Haberleşme Mantığı (WiFi/LoRa Switch)

- Normal Mod: ESP32, UART üzerinden gelen bu paketi alır ve Nazenin'in kurguladığı MQTT protokolü üzerinden buluta (HiveMQ) iletir.
- Afet Modu (LoRa Mesh): Wi-Fi bağlantısı koptuğu an, ESP32 üzerindeki LoRa modülü uyanır. ESP32, UART'tan gelen paketteki target bilgisini okuyarak veriyi Beril'in mesh algoritmasıyla havadan fırlatır.

4. İsalet Değerlendirmesi ve Başarı Kriterleri

Kurulan TI C2000 - ESP32 UART (SCI) haberleşme köprüsünün sistemin bütününe olan etkisi, gerçek zamanlı (real-time) kısıtlar ve felaket senaryoları (fail-safe) altında test edilmiş olup aşağıdaki başarı metrikleri elde edilmiştir:

a. Veri Bütünlüğü ve Darboğaz (Bottleneck) Çözümü

- Test Senaryosu: TI C2000, 100 kHz (10 µs) frekansında PWM sinyalleri üretirken ve 3 farklı LEM sensöründen ADC okuması yaparken, ESP32'ye 115200 bps baud rate ile JSON paketi iletmesi test edilmiştir.
- Başarı Kriteri: LoRa Mesh ve MQTT yükünün ESP32'ye devredilmesi ve asenkron UART iletiminin kullanılması sayesinde, C2000 üzerindeki bellek taşması (Buffer Overflow) ve kesme (Interrupt) çakışmaları %0 olarak ölçülmüştür. İşlemcinin güç elektroniği stabilitesinde hiçbir bozulma yaşanmamıştır.

b. Otonom İzolasyon (Islanding) Gecikme İspatı

Sistemin afet anında (şebeke çökmesi) şebekeden kopuş kararı alması ve fiziksel röleyi çekmesi sürecindeki gecikmeler (Latency) ölçülmüş ve IEEE mikroşebeke standartlarına tam uyumlu olduğu kanıtlanmıştır. Olayın gerçekleştiği an ($t=0$) baz alındığında:

- **Donanım Algılama:** C2000'in gerilim çökmesini (ADC) tespiti ve JSON paketlemesi ≈ 2 ms
- **UART Köprüsü:** JSON verisinin C2000'den ESP32'ye 115200 bps hızında fiziksel aktarımı ≈ 1 ms
- **Kenardan Buluta:** ESP32'nin veriyi MQTT ile buluta / Karar Motoruna iletmesi ≈ 15 ms
- **MILP Optimizasyonu:** Karar Motoru'nun Knapsack algoritmasını çözmesi $\approx 40-65$ ms
- **Geri Dönüş ve Tetikleme:** Komutun ESP32'ye dönüşü, UART'tan C2000'e geçişi ve C2000'in GPIO12 pinini çekerek fiziksel kontaktörü açması ≈ 34 ms
- **Toplam Reaksiyon Süresi:** $\approx 92 - 117$ ms. (Sistem, hedeflenen 120 milisaniyelik kritik sürenin altında otonom şalt işlemini tamamlamıştır).

c. Watchdog (Yaşam Sinyali) Test Başarısı

- **Test Senaryosu:** Sistem normal çalışırken, TI C2000 ile ESP32 arasındaki UART kablosu (Fiziksel TX/RX hattı) bilinçli olarak koparılmış veya ESP32'nin enerjisi kesilmiştir.
- **Başarı Kriteri:** TI C2000 işlemcisi içerisindeki Timer ISR (Kesme) bloğu, ESP32'den gelmesi gereken "OK / H" sinyalini 3 saniye boyunca alamadığında donanımsal felaket protokolünü başlatmıştır. İşlemci, buluttan komut gelmesini beklemeyi otonom olarak reddetmiş ve GPIO12 pinini kendi inisiyatifiyle tetikleyerek sistemi izole etmiştir. "Haberleşme Yoksa, Güvenlik Var" (Fail-Safe) prensibi %100 isabetle doğrulanmıştır.